



Département des Technologies de
l'information et de la communication (TIC)
Informatique et systèmes de communication
Sécurité informatique

Travail de Bachelor

Rapport Intermédiaire

Portage de la pile de communication Nanostack (Wi-SUN) sur
Zephyr RTOS

Étudiant	Benoit Delay
Enseignant responsable	Prof. Favrat Pierre
Année académique	2025-26

Yverdon-les-Bains, le 02.04.2026

Table des matières

1	Glossaire des termes techniques	6
2	Introduction	9
2.1	Structure du document	10
3	Cahier des charges	11
3.1	Résumé du contexte	11
3.1.1	Problématique	11
3.2	Phase du projet	12
3.2.1	Livrables	13
4	Planification	15
4.1	Planification initiale	15
4.2	Diagramme de Gantt initiale	17
5	État de l'art	19
5.1	Standard Wi-SUN	19
5.2	Mbed OS	19
5.3	Zephyr	20
5.3.1	Gestion de l'énergie	20
5.3.2	Connectivité et protocoles	21
5.4	Nanostack	22
5.4.1	SAL (Software Abstraction Layer)	22
5.4.2	HAL (Hardware Abstraction Layer)	23
5.4.3	Le moteur d'événements (<i>Nanostack Event Loop</i>)	23
5.4.4	Dépendance au système d'exploitation hôte (CMSIS-RTOS)	24
5.4.5	Réception des paquets sous Mbed OS	24
5.5	Conclusion de l'état de l'art	26

6 Méthodologie et architecture du portage	27
6.1 Intégration logicielle : Bibliothèque vs Module Zephyr	27
6.1.1 L'approche Bibliothèque classique	27
6.1.2 L'approche Module Zephyr (<i>Out-of-Tree</i>)	27
6.1.3 Comparaison des approches	28
6.1.4 Décision architecturale et Manifeste west	29
6.2 Implémentation de la structure du module	29
6.3 Adaptation du moteur d'événements (<i>Event Loop</i>)	30
6.4 Intégration cryptographique (Mbed TLS)	30
6.5 Refonte des règles de compilation (CMakeLists)	31
7 Conclusion du rapport intermédiaire	33
7.1 Bilan et planification des travaux futurs	33
7.2 État des lieux de la planification	34
Bibliographie	35

1 Glossaire des termes techniques

Acronyme / Terme	Définition
RTOS	Real-Time Operating System : Système d'exploitation déterministe conçu pour traiter un événement et fournir une réponse dans un délai strict et garanti.
SAL	Software Abstraction Layer : Couche logicielle de la Nanostack qui offre une interface standardisée (API Socket) et gère les protocoles réseau indépendamment du matériel.
HAL	Hardware Abstraction Layer : Couche d'abstraction matérielle. Elle fait le lien entre le logiciel (comme la SAL) et les composants physiques (antenne radio, timers, etc.).
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks : Protocole permettant de compresser et transmettre des paquets IPv6 sur des réseaux sans fil à faible consommation (comme le 802.15.4).
RPL	Routing Protocol for Low-Power and Lossy Networks : Protocole de routage dynamique utilisé par la Nanostack pour trouver le meilleur chemin dans un réseau maillé (mesh).
Mbed TLS	Bibliothèque de cryptographie autonome (anciennement liée à Mbed OS) fournissant les mécanismes de sécurité comme TLS et DTLS pour chiffrer les communications.
Smart City (Ville intelligente)	Ville utilisant les technologies de communication sans fil afin de mieux gérer ses ressources et ses infrastructures.
IoT	Internet of things, l'internet des objets est le réseau connectant l'internet et les objets connectés

Tableau 1. – Glossaire des termes techniques

2 Introduction

L'émergence des Smart Cities transforme radicalement la gestion des infrastructures urbaines en imposant une connectivité omniprésente et en temps réel. Ce changement d'échelle crée un besoin critique pour des réseaux capables de s'affranchir des contraintes physiques de la ville, notamment en termes de densité et de portée. Les solutions traditionnelles atteignent leurs limites face à des déploiements massifs, lorsqu'il s'agit de connecter des dizaines de milliers d'appareils, allant des compteurs électriques intelligents à l'éclairage public.

Pour répondre à ces problématiques, le standard Wi-SUN a été créé en janvier 2012. Il s'appuie sur une couche physique sub-GHz (norme IEEE 802.15.4g), offrant une meilleure propagation radio en milieu urbain dense et une grande résilience face aux obstacles physiques. Ce réseau maillé permet d'interconnecter des millions d'appareils grâce à l'adressage IPv6 et au protocole de routage RPL (Routing Protocol for Low-power and Lossy Networks)[1]. Au-delà de ses capacités d'auto-découverte et d'auto-réparation, il garantit la sécurité du réseau via le standard IEEE 802.1X[2], qui assure l'authentification de chaque nœud. Enfin, la Wi-SUN Alliance encadre l'interopérabilité entre les constructeurs, garantissant ainsi l'homogénéité et la pérennité des déploiements.

Zephyr est un RTOS en plein essor, soutenu par la Linux Foundation depuis sa première publication en février 2016 [3]. Il est conçu pour des appareils et des systèmes présentant de fortes contraintes mémoire. Son architecture repose sur un unique espace d'adressage ainsi qu'une gestion statique de la mémoire. Il reprend des concepts essentiels à Linux, tels que le DeviceTree et Kconfig, ce qui le rend hautement configurable et facilement adaptable à de nouvelles cibles matérielles. Néanmoins, à l'heure actuelle, il n'existe aucune implémentation open-source du standard Wi-SUN sur ce RTOS, malgré le fait qu'il intègre déjà nativement de nombreux protocoles réseau.

Nanostack est une pile de communication open-source intégrant le standard Wi-SUN. Actuellement, elle est étroitement liée à l'écosystème Mbed OS[4], un système qui n'est plus maintenu depuis 2024[5]. Ce travail de Bachelor a pour but de pallier cette obsolescence en portant la Nanostack vers Zephyr et en l'intégrant pleinement à son architecture. À terme,

cette contribution permettra de déployer Zephyr dans un contexte de Smart Cities de manière entièrement open-source, tout en garantissant la pérennité de cette solution réseau.

2.1 Structure du document

Ce rapport intermédiaire s'articule autour des axes suivants :

- **Cahier des charges** : Définition des phases du projet et des livrables attendus.
- **Planification initiale** : Présentation du calendrier prévisionnel et des jalons du projet.
- **État de l'art** : Analyse des différentes technologies étudiées en vue du portage.
- **Stratégie de portage de la Nanostack** : Détail de la méthodologie employée pour adapter les différents composants, ainsi que l'argumentaire des choix techniques réalisés.
- **Bilan et planification actualisée** : État d'avancement des travaux au regard de la planification initiale et ajustement pour la suite du projet.

3 Cahier des charges

3.1 Résumé du contexte

L'essor des villes intelligentes (*Smart Cities*) rend indispensable le déploiement de réseaux capables d'interconnecter une multitude d'équipements. Ces applications imposent des contraintes réseau strictes : une densité élevée d'appareils, une couverture longue portée et une forte résilience face aux obstacles urbains.

Pour répondre aux défis de l'IoT (Internet des Objets), **Zephyr** s'est imposé comme une référence. Ce système d'exploitation temps réel (RTOS), lancé en 2016, est spécialement conçu pour les petits systèmes embarqués connectés et à faibles ressources. Il offre une connectivité avancée, de multiples API de communication et prend en charge un large éventail d'architectures 32 et 64 bits.

De son côté, la **Nanostack** est une pile de communication open source reconnue pour son intégration robuste des réseaux maillés, notamment via le protocole Wi-SUN. Cependant, elle a été historiquement développée pour le système d'exploitation Mbed OS, dont la maintenance officielle a pris fin au profit d'autres solutions.

3.1.1 Problématique

Mbed OS n'étant plus supporté, le projet consiste à porter la pile réseau Nanostack vers Zephyr RTOS afin de pérenniser son utilisation. L'enjeu technique principal réside dans l'adaptation de la couche d'abstraction matérielle (HAL) de la Nanostack pour qu'elle puisse interagir nativement avec l'écosystème et les pilotes matériels de Zephyr.

Ce portage s'articulera autour de deux phases majeures :

1. **Fonctionnement autonome (*Standalone*)** : Adapter et faire tourner le cœur de la Nanostack sur Zephyr de manière indépendante, en validant les interactions de bas niveau (radio, timers, gestion mémoire).

2. **Intégration système(Optional)** : Coupler intimement la Nanostack à la pile réseau native de Zephyr, afin qu'elle puisse être exploitée par les applications de haut niveau de manière totalement transparente, via l'API standard des *sockets* BSD.

3.2 Phase du projet

1. Prise en main de la base de code
 - Fork les sources de mbed-os
 - Fork les sources de zephyr-os
 - Prise en main de la hiérarchie des dossiers de la Nanostack
 - Mise en évidence des différents dossiers et leur utilisation
2. Prise en main de Zephyr OS
 - Installation de la toolchain de Zephyr
 - Compiler le kernel ainsi qu'un exemple sur qemu
 - Compiler le kernel ainsi qu'un exemple sur une cible
 - Ecrire un petit programme de test
3. Mettre en évidence les différences de fonctionnement entre mbed-os et zephyr
 - Comprendre comment fonctionne la event-loop de mbed-os
 - Comprendre comment fonctionne les interfaces reseau de mbed-os
 - Comprendre comment fonctionne les driver réseau sur zephyr
 - Isoler les différents composants
4. Ecrire le rapport intermediaire
 - Documenter l'avancement des travaux de recherche préliminaires
 - Rédiger une étude théorique sur les mécanismes internes de Zephyr (gestion de l'énergie, architecture réseau, etc.)
 - Établir les choix techniques et méthodologiques pour la phase d'implémentation
5. Isoler la partie « métier » de la stack et isoler les fonctions liées au kernel
 - Copier la partie qui n'est pas reliées à l'os
 - Commencer à faire l'inventaire des fonctions à porter
6. Ecrire la HAL(Hardware Abstraction Layer)
 - Remplacer toute les fonctions kernel appelées par la stack par des fonctions de la HAL
 - implémenter chacune de ces fonctions pour zephyr
7. Tester l'implémentation
 - Ecrire des test pour garantir le bon fonctionnement de la stack

- Ecrire un petit programme zephyr qui va permettre de faire communiquer 2 boards entre elles avec la nanostack
8. Optionnel: Intégrer la nanostack à l'écosystème réseau de Zephyr
 - Intégrer la Nanostack au sous-système réseau de Zephyr afin de permettre aux applications de communiquer via l'API standardisée des sockets BSD.
 9. Redaction du rapport final
 - Documenter les implementations
 - Documenter l'avancement du projet

3.2.1 Livrables

Les livrables seront les suivants :

- Le rapport complet
- Un repo Github contenant le portage de la Nanostack
- Un repo Github contenant un code d'exemple

4 Planification

4.1 Planification initiale

Phase	Tâches associées	Temps alloué	Proportion
1. Base de code	- Fork de mbed-os et zephyr-os - Analyse de la hiérarchie de la Nanostack	20 h	4%
2. Prise en main Zephyr	- Installation de la toolchain - Compilation kernel + exemples - Programme de test	40 h	9%
3. Étude comparative	- Analyse de l'évent-loop - Interfaces et drivers réseau - Isolation des composants	50 h	11%
4. Rapport intermédiaire	- Rédaction de l'état d'avancement - Explication théorique	20 h	4%

Phase	Tâches associées	Temps alloué	Proportion
5. Isolation métier/kernel	• Copie de la partie agnostique - Inventaire des fonctions à porter	50 h	11%
6. Création de la HAL	• Remplacement des appels kernel - Implémentation pour zephyr	120 h	27%
7. Phase de tests	• Écriture des tests de validation - Dev d'un programme de communication P2P	80 h	18%
8. Intégration (Optionnel)	• Portage pour appel via les sockets BSD	40 h	9%
9. Rapport final	• Documentation des implémentations - Bilan du projet	30 h	7%
Total estimé		450 h	100%

4.2 Diagramme de Gantt initiale

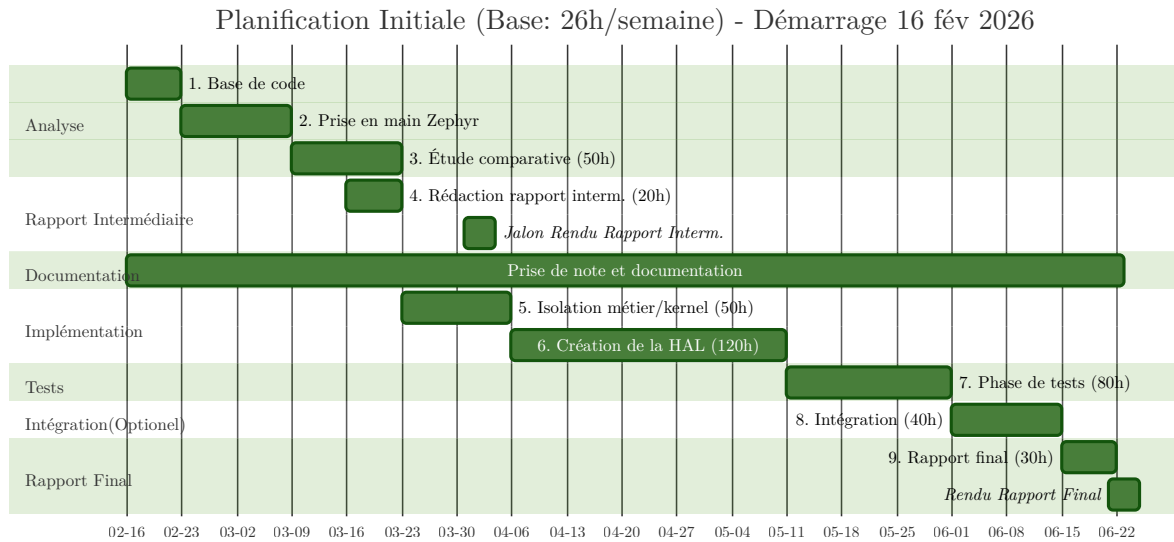


Fig. 1. – Planification Initiale

5 État de l'art

5.1 Standard Wi-SUN

Le standard Wi-SUN a été développé en janvier 2012 par la Wi-SUN Alliance. Il répond à l'émergence des villes intelligentes (*Smart Cities*), qui exigent de pouvoir interconnecter des centaines de milliers, voire des millions d'appareils au sein d'environnements urbains denses et remplis d'obstacles physiques.

Pour garantir une communication fiable en s'affranchissant des problèmes de propagation d'ondes et d'interférences, Wi-SUN s'appuie sur des spécifications optimisées pour cet usage, telles que la couche physique IEEE 802.15.4g (sub-GHz). Au niveau réseau, il utilise un adressage IPv6 couplé au protocole de routage dynamique RPL (*Routing Protocol for Low-Power and Lossy Networks*). L'ensemble de cette architecture a été pensé avec un objectif majeur : minimiser drastiquement la consommation énergétique des nœuds du réseau.

5.2 Mbed OS

Mbed OS est un système d'exploitation temps réel destiné aux systèmes embarqués. Il repose sur CMSIS-RTOS[6], une couche d'abstraction fournie par Arm, qui permet aux développeurs de dialoguer avec le processeur de manière uniforme, indépendamment du fabricant du matériel. Il intègre un noyau temps réel performant ainsi qu'une gestion complète du *multithreading*.

Particulièrement populaire dans l'écosystème de l'IoT, Mbed OS embarque nativement un grand nombre de modules de connectivité. Il fournit des implémentations de haut niveau pour de nombreux protocoles de communication, tout en supportant un vaste catalogue de cartes de développement et de pilotes (*drivers*).

C'est au sein de cet écosystème qu'a été développée la Nanostack, une pile de communication intégrant, entre autres, le standard Wi-SUN. Cependant, Arm a annoncé la fin de vie (*End of Life - EoL*) de Mbed OS pour juillet 2026. Cette obsolescence programmée motive

directement ce projet : porter la Nanostack vers un RTOS moderne et pérenne afin de continuer à exploiter les avantages du standard Wi-SUN.

5.3 Zephyr

Zephyr est un système d'exploitation open source soutenu par la Linux Foundation, dont la version 1.0 a été publiée le 17 février 2016. Il est spécifiquement conçu pour les systèmes embarqués aux ressources limitées répondant à des contraintes temps réel. S'inspirant fortement de Linux, il en reprend des concepts standards tels que le DeviceTree et Kconfig.

C'est un OS extrêmement modulaire qui prend en charge un vaste panel d'architectures matérielles. Pour simplifier le flux de travail, il s'appuie sur son propre outil en ligne de commande, west[7]. Ce dernier permet de cross-compiler, de tester et de déployer aisément son code sur l'ensemble des plateformes compatibles.

5.3.1 Gestion de l'énergie

L'un des objectifs premiers du standard Wi-SUN étant de minimiser la consommation électrique des différents nœuds, il est primordial d'intégrer rigoureusement la gestion de l'énergie lors du portage de la Nanostack. Zephyr permet de contrôler cette consommation à deux niveaux distincts :

5.3.1.1 Gestion globale du système (*System Power Management*)

La gestion de la consommation énergétique au niveau du processeur (CPU) tourne autour de deux politiques de mise en veille principales[8] :

- **Politique basée sur la résidence (*Residency-based*)** : C'est l'approche automatisée par défaut. Le noyau calcule le temps d'inactivité prévu avant la prochaine tâche planifiée. Il sélectionne ensuite automatiquement l'état de veille le plus profond possible, à condition que le temps d'inactivité soit supérieur au temps de résidence (le seuil de rentabilité énergétique de cet état). Les différents états de veille supportés par le matériel et leurs caractéristiques temporelles sont définis de manière statique dans le *DeviceTree* à l'aide des *bindings* `zephyr,power-state`.
- **Politique définie par l'application (*Application-defined*)** : Dans cette configuration, le noyau délègue la prise de décision. C'est au développeur d'implémenter sa propre logique métier pour basculer manuellement le CPU dans les modes d'économie d'énergie adéquats, offrant ainsi un contrôle total basé sur les événements et le comportement spécifique de l'application.

5.3.1.2 Gestion de l'alimentation des périphériques (*Device Power Management*)

Le *Device Power Management* confie la gestion de la consommation énergétique directement aux pilotes (*drivers*) des périphériques. Zephyr propose deux méthodes distinctes pour contrôler l'état d'alimentation de ces composants matériels[9] :

- **Gestion dynamique (*Device Runtime Power Management*)** : Dans ce modèle asynchrone, l'état d'alimentation d'un périphérique est géré de manière autonome en fonction de son utilisation réelle. Les pilotes peuvent adapter leur consommation, mais ils peuvent également être contrôlés explicitement par les applications de haut niveau ou les autres sous-systèmes de Zephyr via des API dédiées, permettant un réveil « à la demande ».
- **Gestion centralisée par le système (*System-Managed Device Power Management*)** : Dans ce mode, le noyau coordonne la mise en veille des périphériques en synchronisation avec celle du processeur (CPU). Lorsqu'une transition vers un état de basse consommation global est décidée, le système se charge de suspendre automatiquement tous les périphériques inactifs. Cette suspension s'effectue strictement dans l'ordre inverse de leur initialisation, afin de préserver les dépendances matérielles et de garantir la stabilité du système.

5.3.2 Connectivité et protocoles

Pour répondre aux exigences de l'Internet des Objets, Zephyr embarque nativement une pile réseau complète, modulaire et hautement optimisée, appelée le *Net Subsystem*. Cette pile est conçue pour s'adapter aux architectures à faibles ressources tout en offrant des fonctionnalités dignes des systèmes d'exploitation majeurs.

L'architecture réseau de Zephyr fonctionne avec plusieurs couches clé :

- **L'interface applicative (API)** : Zephyr expose une interface de programmation basée sur le standard POSIX (*BSD Sockets*). Ce choix architectural est crucial, car il permet aux développeurs de créer des applications réseau de haut niveau (utilisant CoAP, MQTT ou HTTP) de manière totalement agnostique vis-à-vis du matériel et des protocoles sous-jacents.
- **Le cœur réseau (Couches 3 et 4)** : Le système gère nativement le routage IPv4 et IPv6, ainsi que les protocoles de transport TCP et UDP.
- **La couche liaison de données (Couche 2)** : Le *Net Subsystem* supporte une grande variété de technologies physiques et de protocoles de liaison, tels que l'Ethernet, le Wi-Fi, le Bluetooth Low Energy (BLE) et la norme radio IEEE 802.15.4.

Bien que Zephyr intègre nativement la norme IEEE 802.15.4 et supporte des protocoles maillés comme Thread (via l'intégration d'OpenThread), il souffre actuellement d'un manque important pour les déploiements urbains à grande échelle : il n'existe à ce jour aucune implémentation open source native du standard **Wi-SUN** dans son écosystème.

C'est précisément cette lacune que le portage de la Nanostack vient combler.

5.4 Nanostack

La Nanostack est la pile réseau développée originellement pour Mbed OS. Mbed OS arrivant en fin de vie, la migration de cette pile logicielle vers un nouvel OS est devenue indispensable.

Pour mener à bien ce portage, il est important d'appréhender l'architecture interne de la Nanostack. Celle-ci repose sur une séparation stricte des responsabilités, divisée en deux couches principales, animées par un moteur d'événements :

- La **SAL** (*Software Abstraction Layer*) : La partie purement logicielle et applicative.
- La **HAL** (*Hardware Abstraction Layer*) : L'interface avec les composants matériels.
- Le **Nanostack Event Loop** : Le cœur asynchrone qui cadence l'ensemble des opérations.

5.4.1 SAL (Software Abstraction Layer)

La SAL gère toute la partie purement logicielle de la Nanostack, totalement indépendante du matériel (qui est délégué à la couche matérielle, la HAL). Son rôle principal est de masquer la complexité du réseau en offrant une interface de programmation standardisée (API Socket) à l'application.

C'est dans cette couche que s'exécutent les machines d'état des différents protocoles (MAC, 6LoWPAN, routage RPL, IPv6, TCP/UDP). Pour fonctionner efficacement sur des microcontrôleurs limités, la SAL s'appuie sur trois piliers :

- **Le Nanostack Event Loop** : Un ordonnanceur d'événements coopératif qui gère le trafic réseau de façon asynchrone pour économiser la RAM, sans nécessiter de multiples *threads*.
- **Une gestion interne du temps et de la mémoire** (`ns_timer` et `ns_dyn_mem`) : Un système de minuteurs virtuels pour les *timeouts* réseau et un allocateur de *buffers* optimisé pour prévenir la fragmentation de la mémoire lors d'un trafic intense.

Enfin, la SAL intègre nativement les mécanismes de sécurité (via Mbed TLS) pour chiffrer les communications avant leur transmission. Il est important de noter que bien que Mbed OS ait atteint sa fin de vie, le projet Mbed TLS demeure activement maintenu par la

communauté TrustedFirmware [10]. Son intégration reste donc pertinente et pérenne pour ce projet, nous dispensant de chercher une solution cryptographique alternative.

5.4.2 HAL (Hardware Abstraction Layer)

La HAL fait le pont entre le système d'exploitation hôte (Zephyr) et la Nanostack. Son rôle est de relier la logique logicielle de la pile réseau aux composants physiques. C'est à ce niveau que se fait l'interface avec les pilotes matériels (*drivers*), indispensables pour communiquer via Ethernet ou via la norme IEEE 802.15.4.

Pour réussir le portage vers Zephyr, l'implémentation de la HAL doit couvrir plusieurs domaines critiques :

- **L'allocation de la mémoire** : Bien que la Nanostack possède son propre gestionnaire de mémoire interne, elle doit interagir avec le noyau du RTOS hôte lors de son initialisation pour réserver son bloc de mémoire principal (via l'allocation dynamique de Zephyr ou l'assignation d'un bloc statique).
- **La gestion de la concurrence (Multithreading)** : Mbed OS et Zephyr étant tous deux des RTOS, la HAL doit traduire les primitives de synchronisation. Il est impératif d'implémenter les mutex, les sémaphores et les sections critiques pour protéger l'état de la Nanostack contre les accès concurrents provenant de différents *threads*.
- **Les horloges matérielles et interruptions** : La HAL doit faire le lien entre les minuteurs (*timers*) matériels gérés par Zephyr et les événements temporels de la Nanostack, tout en gérant les interruptions matérielles (par exemple, signaler à la pile logicielle qu'un paquet radio vient d'être physiquement reçu).

5.4.3 Le moteur d'événements (*Nanostack Event Loop*)

Au cœur de l'architecture de la Nanostack se trouve un composant central : le *Nanostack Event Loop*. Plutôt que de créer un fil d'exécution (*thread*) pour chaque connexion ou tâche réseau, ce qui serait bien trop coûteux en mémoire pour un microcontrôleur, la pile utilise un ordonnanceur d'événements coopératif.

Toutes les actions du réseau (réception d'un paquet, expiration d'un délai, demande de l'application) sont converties en événements et placées dans une file d'attente globale (*Event Queue*). L'Event Loop dépile ensuite ces événements un par un de manière asynchrone et déclenche les fonctions de rappel (*callbacks*) associées. Ce fonctionnement garantit une utilisation minimale et déterministe de la RAM.

5.4.4 Dépendance au système d'exploitation hôte (CMSIS-RTOS)

Bien que la Nanostack soit agnostique vis-à-vis du matériel, elle dépend intrinsèquement des services du système d'exploitation sur lequel elle tourne pour ses opérations fondamentales. Dans son environnement d'origine (Mbed OS), cette dépendance est gérée via l'API standardisée **CMSIS-RTOS**.

La pile réseau fait régulièrement appel à cette couche de compatibilité pour trois fonctions critiques :

- **La synchronisation** : Utilisation de verrous (*mutexes*) pour protéger ses structures de données internes lorsque des requêtes proviennent de différents *threads* applicatifs.
- **L'allocation mémoire** : Réservation de la mémoire au démarrage pour son allocateur interne (*ns_dyn_mem*).
- **La gestion du temps** : Interfaçage avec les horloges du système pour la gestion des *timeouts* et le cadencement de l'Event Loop.

Enjeu pour le portage : C'est sur ce point précis que se situe le cœur de la migration. Les appels à CMSIS-RTOS étant profondément ancrés dans la couche d'adaptation de la Nanostack, l'objectif du projet sera de substituer ces appels par les primitives natives de Zephyr (comme les API *k_mutex* ou *k_timer*), assurant ainsi une intégration fluide sans dénaturer le code source des protocoles.

5.4.5 Réception des paquets sous Mbed OS

Pour comprendre les enjeux du portage, il est essentiel d'analyser le flux de données actuel lors de la réception d'un paquet sous Mbed OS. Mbed OS étant majoritairement orienté objet (C++), tandis que la Nanostack est écrite en C, le passage du monde matériel au monde réseau nécessite une interface de traduction.

Le flux de réception d'une trame radio (ex: IEEE 802.15.4) se déroule en quatre étapes :

1. **L'interruption matérielle (ISR)** : Lorsque la puce radio reçoit un paquet valide, elle lève une interruption matérielle. Le noyau Mbed OS intercepte cette IRQ et exécute la routine d'interruption du pilote radio.
2. **L'interface matérielle (NanostackRfPhy)** : Mbed OS utilise une classe abstraite en C++ nommée *NanostackRfPhy*. Le pilote radio spécifique à la carte (par exemple, un driver Atmel ou STM32) hérite de cette classe. C'est ce pilote qui va lire les données brutes sur le bus matériel (SPI/UART).
3. **Le pont C/C++ et la fonction de rappel (Callback)** : Lors de l'initialisation du système, la classe *NanostackRfPhy* enregistre le périphérique radio auprès de la couche C de la Nanostack via l'API *arm_net_phy_register()*. Cette fonction fournit au pilote un pointeur vers

une fonction de réception interne à la pile (souvent `phy_rx_cb`). Le pilote Mbed OS appelle cette fonction C en lui passant les données du paquet.

4. **L'Event Loop dans un Thread Mbed OS** : C'est ici que la magie de la Nanostack opère. La fonction de rappel ne traite pas la trame directement. Elle alloue un tampon avec `ns_dyn_mem`, y copie la charge utile, et poste un événement réseau dans le *Nanostack Event Loop*. Sous Mbed OS, cet *Event Loop* s'exécute en continu à l'intérieur d'un *thread CMSIS-RTOS* dédié (souvent nommé `ns_thread`). Le RTOS réveille ce *thread*, qui dépile l'événement et fait remonter le paquet de manière asynchrone vers les couches MAC, 6LoWPAN et IPv6.

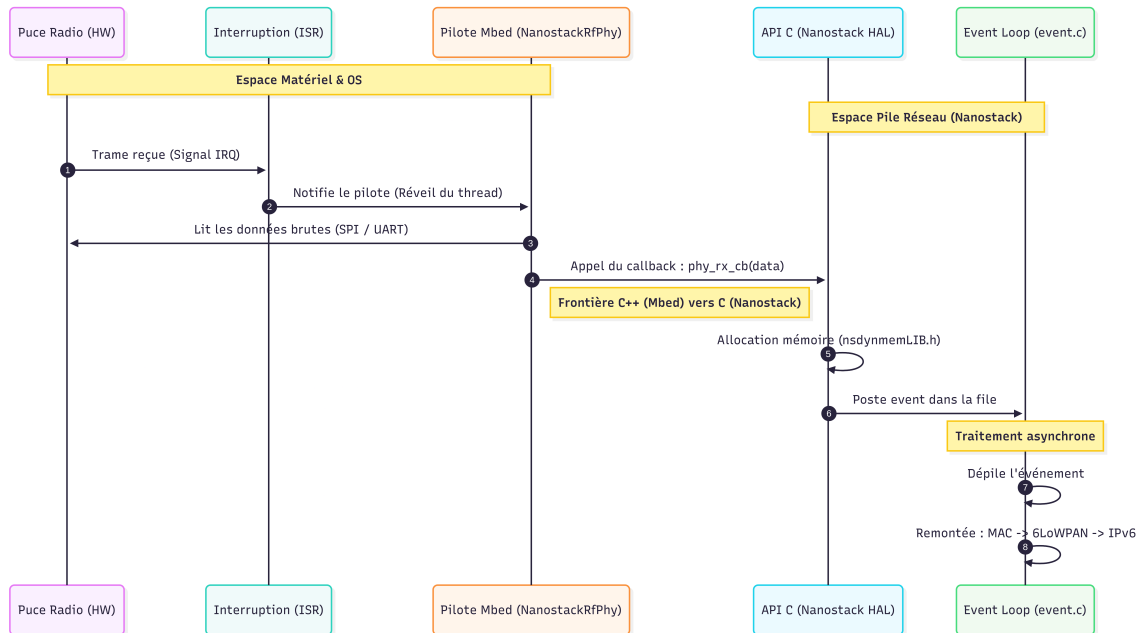


Fig. 2. – Transaction driver -> Nanostack

L'architecture actuelle démontre que la Nanostack n'a aucune conscience de la classe C++ `NanostackRfPhy` ni du fonctionnement interne de l'ISR de Mbed OS. Elle s'attend uniquement à ce qu'un code externe (la HAL) appelle sa fonction de rappel C (`phy_rx_cb`) et lui fournisse des événements. Dans Zephyr, il suffira de reproduire ce comportement depuis l'API `ieee802154 native`.

5.5 Conclusion de l'état de l'art

L'analyse de ces différentes technologies met en évidence la pertinence du portage de la Nanostack vers Zephyr. D'un côté, le standard Wi-SUN répond parfaitement aux exigences des réseaux urbains modernes. De l'autre, Zephyr offre une base temps réel robuste, modulaire et taillée pour l'efficacité énergétique, mais manque encore d'une implémentation Wi-SUN native. L'architecture interne de la Nanostack, séparant habilement la logique réseau (SAL) des interactions matérielles (HAL), rend cette intégration réalisable. Le défi technique ne réside donc pas dans la réécriture des protocoles, mais bien dans la conception des interfaces (adaptateurs) entre la HAL de la Nanostack et les API natives de Zephyr.

6 Méthodologie et architecture du portage

6.1 Intégration logicielle : Bibliothèque vs Module Zephyr

Lors du portage d'une base de code aussi volumineuse que la Nanostack, la méthode d'intégration au système de compilation est un choix d'architecture critique. Zephyr propose deux approches principales pour inclure du code tiers : l'intégration en tant que bibliothèque statique au sein de l'application, ou la création d'un Module Zephyr externe [11].

6.1.1 L'approche Bibliothèque classique

Cette méthode consisterait à copier l'intégralité des fichiers sources de la Nanostack directement dans le dossier de l'application finale, et à configurer le fichier `CMakeLists.txt` de l'application pour les compiler.

- **Avantage** : Mise en place initiale très rapide.
- **Inconvénient** : Cette approche lie fortement la pile réseau à l'application métier. Elle rend la mise à jour de la Nanostack difficile, pollue l'historique de version (Git) du projet principal et rend la réutilisation de la pile par d'autres projets quasiment impossible.

6.1.2 L'approche Module Zephyr (*Out-of-Tree*)

Un Module Zephyr est un dépôt de code (par exemple hébergé sur GitHub) totalement indépendant de l'arbre source de Zephyr et de l'application finale. Il est automatiquement téléchargé et lié lors de la configuration du projet grâce à l'outil de gestion de paquets `west` (via le fichier `west.yml`).

- **Séparation des responsabilités** : Le code de la Nanostack reste isolé. L'application métier se contente de l'appeler via les API publiques.
- **Intégration native (Kconfig et CMake)** : Un module permet de définir ses propres menus de configuration (Kconfig). Ainsi, les paramètres de la Nanostack (activation du Wi-

SUN, taille des *buffers*) apparaîtront directement dans les menus de configuration de Zephyr (via `menuconfig`), exactement comme les sous-systèmes natifs.

- **Portabilité et partage** : En plaçant la Nanostack dans son propre dépôt Git structuré comme un module, elle devient facilement distribuable à l'ensemble de la communauté open source.

6.1.3 Comparaison des approches

Afin de justifier le choix de l'architecture logicielle, le tableau suivant dresse le bilan des avantages et inconvénients des deux méthodes d'intégration au sein de l'écosystème Zephyr :

Approche	Avantages	Inconvénients
Bibliothèque classique (<i>In-Tree</i>)	• Mise en place initiale rapide et intuitive. • Ne requiert pas d'outils externes (uniquement CMake). • Utile pour un prototypage très basique et isolé.	• Forte dépendance logicielle (couplage fort) entre l'application et la pile réseau. • Mise à jour future de la Nanostack très complexe. • Pollution de l'historique Git du projet principal. • Aucune intégration native avec les menus de configuration Kconfig. • Réutilisation par d'autres projets plus ardue.
Module Zephyr (<i>Out-of-Tree</i>)	• Séparation stricte des responsabilités. • Intégration native aux menus Kconfig de Zephyr. • Gestion des dépendances automatisée via l'outil <code>west</code>. • Versionnement Git totalement indépendant. • Code standardisé et facilement distribuable.	• Courbe d'apprentissage initiale légèrement plus rude (nécessite de maîtriser le format <code>module.yml</code>). • Configuration initiale des fichiers de liaison plus verbeuse.

Tableau 2. – Comparaison des méthodes d'intégration logicielle sous Zephyr

6.1.4 Décision architecturale et Manifeste `west`

Pour ce projet de Bachelor, **l'approche par Module Zephyr est retenue**. Elle est la seule à garantir une architecture logicielle pérenne. Le choix de cette architecture en module externe (*Out-of-Tree*) implique l'utilisation avancée du système de manifeste.

L'application finale contiendra un fichier `west.yml`. Lors de l'initialisation de l'environnement (via la commande `west update`), l'outil se chargera de cloner le dépôt de la Nanostack, de l'insérer dans l'espace de travail (*workspace*), et de parser son fichier `zephyr/module.yml` pour lier ses menus et scripts de compilation au système principal.

6.2 Implémentation de la structure du module

Pour être reconnu par le système de compilation de Zephyr, le module externe doit respecter une topologie stricte et fournir trois fichiers fondamentaux : `module.yml` (description), `CMakeLists.txt` (compilation) et `Kconfig` (configuration).

La topologie classique du module développé pour ce portage est la suivante :

```
zephyr-nanostack/
├── zephyr/
│   └── module.yml    <-- Déclaration du module pour west
├── include/         <-- En-têtes publics (API de la Nanostack)
├── src/              <-- Code source C (event_loop, adaptateurs...)
├── CMakeLists.txt    <-- Règles de compilation Zephyr
└── Kconfig           <-- Options d'activation pour le menuconfig
```

Liste 1. – Arborescence type du module Zephyr pour la Nanostack

Côté application, il suffira de modifier le fichier `west.yml` du projet cible pour y indiquer cette nouvelle dépendance et son URL Git :

```
1 manifest:
2 projects:a
3   - name: zephyr-nanostack
4     url: [https://github.com/Travail-de-Bachelor-Delay-Benoit/zephyr-nanostack.git](https://github.com/Travail-de-Bachelor-Delay-Benoit/zephyr-nanostack.git)
5     revision: main
```

Listing 2. – Déclaration du module dans le `west.yml` de l'application

Une fois le module téléchargé, l'activation de la pile réseau se fera via le système de configuration standard. Le fichier Kconfig du module exposera les options nécessaires :

```
1 menuconfig NANOSTACK
2     bool "Support de la pile réseau Nanostack (Wi-SUN)"
3     default n
4     help
5         Active la compilation et l'intégration de la Nanostack.
```

Listing 3. – Fichier Kconfig à la racine du module

Enfin, le développeur n'aura plus qu'à ajouter `CONFIG_NANOSTACK=y` dans le fichier `prj.conf` de son application pour que la pile soit automatiquement incluse à la compilation.

6.3 Adaptation du moteur d'événements (*Event Loop*)

L'un des défis majeurs du portage consiste à substituer les primitives de Mbed OS (CMSIS-RTOS) par celles de Zephyr. Le cœur de la Nanostack reposant sur un moteur d'événements (*Event Loop*), ce dernier doit s'exécuter dans son propre fil d'exécution (*thread*).

Sous Zephyr, ce fil d'exécution sera instancié à l'aide de l'API native `K_THREAD_DEFINE` ou `k_thread_create`. Il est crucial que l'implémentation de ce *thread* prenne rigoureusement en compte la politique de gestion de l'énergie. En effet, la basse consommation étant l'un des piliers du standard Wi-SUN, la boucle d'événements ne doit en aucun cas monopoliser le processeur. Lorsqu'aucun événement réseau n'est présent dans la file d'attente, le *thread* de la Nanostack devra explicitement relâcher la main (via `k_sleep` ou en bloquant sur une primitive de synchronisation) afin de permettre à Zephyr de basculer le système en veille profonde (*Deep Sleep*).

6.4 Intégration cryptographique (Mbed TLS)

Le standard Wi-SUN impose des exigences de sécurité strictes, nécessitant l'authentification des nœuds (EAP-TLS) et le chiffrement des trames. La Nanostack s'appuie historiquement sur le projet **Mbed TLS** pour sa partie cryptographique.

Cette dépendance s'intègre parfaitement à la stratégie de portage puisque Zephyr utilise également Mbed TLS comme bibliothèque cryptographique officielle par défaut. Le portage ne nécessitera donc pas de réécrire les algorithmes de sécurité. Il consistera principalement à s'assurer que le système de compilation du module (via CMake et Kconfig) force l'activation de Mbed TLS (`CONFIG_MBEDTLS=y`).

De plus, une attention particulière devra être portée sur l'entropie matérielle. Pour générer des clés cryptographiques robustes, la pile devra être correctement interfacée avec le générateur de nombres aléatoires matériel (*True Random Number Generator* - TRNG) exposé par les API de Zephyr.

6.5 Refonte des règles de compilation (CMakeLists)

La dernière étape stratégique concerne l'adaptation du processus de compilation. Mbed OS utilisait son propre système d'assemblage, qui doit être entièrement remplacé par des directives CMake compatibles avec l'écosystème Zephyr.

Le fichier CMakeLists.txt situé à la racine du module exploitera les macros spécifiques de l'OS (comme `zephyr_library()` et `zephyr_library_sources()`). Ces directives permettront d'inclure conditionnellement les fichiers sources en langage C de la Nanostack uniquement si l'option `CONFIG_NANOSTACK` est sélectionnée par l'utilisateur, garantissant ainsi que le code mort n'encombre pas la mémoire flash des applications ne nécessitant pas le Wi-SUN.

De plus, une attention particulière devra être accordée à la compilation du cœur de la Nanostack (les parties totalement agnostiques et indépendantes du système d'exploitation). Il sera également impératif de refléter avec précision la nouvelle arborescence des fichiers dans les directives CMake, afin de garantir la bonne résolution des chemins d'inclusion (*headers*) et de préserver l'intégrité de l'architecture logicielle.

7 Conclusion du rapport intermédiaire

7.1 Bilan et planification des travaux futurs

La première phase de ce travail de Bachelor a permis de mener à bien l'analyse exhaustive du code source de la Nanostack et de s'appropriier les concepts avancés de l'écosystème Zephyr RTOS. Cette étude théorique et la définition de l'architecture logicielle posent les bases nécessaires à la réalisation technique du projet.

La seconde phase se concentrera sur l'implémentation pratique de la stratégie détaillée précédemment, à savoir le portage effectif de la Nanostack sous forme de module Zephyr. Une fois cette intégration logicielle achevée, une phase de validation matérielle sera menée. Elle consistera à développer des programmes de test spécifiques et à les déployer sur les cartes de développement, afin d'évaluer le fonctionnement du portage.

Enfin, en fonction du temps imparti à l'issue de la phase de test, une étape d'intégration système avancée sera étudiée. Celle-ci aura pour but d'interfacer directement la Nanostack avec le sous-système réseau natif de Zephyr (*Net Subsystem*). L'accomplissement de cette tâche permettrait aux applications d'interagir avec le réseau Wi-SUN de manière totalement transparente, via l'API standardisée des *sockets* BSD.

7.2 État des lieux de la planification

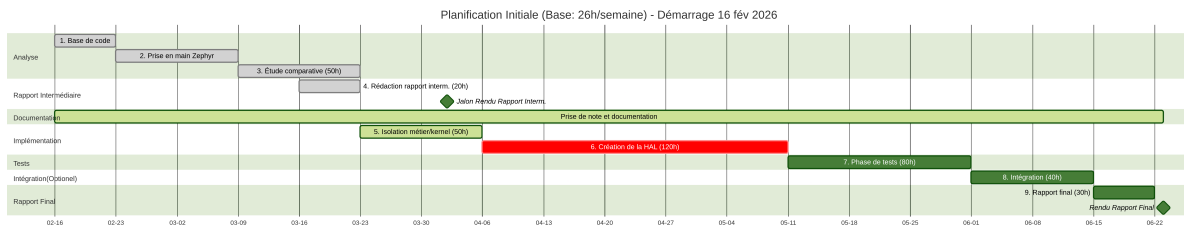


Fig. 3. – État de la planification au 02.04.2026

Sur le plan du calendrier, l'avancement global du projet est conforme aux prévisions. Bien que la rédaction de ce rapport intermédiaire ait nécessité un investissement en temps supérieur aux estimations initiales, ce décalage a été entièrement compensé par une réalisation plus rapide de l'étude comparative.

Concernant les étapes à venir, une légère révision de la charge de travail a été effectuée. La phase de portage nécessitera probablement davantage de temps que prévu, notamment en raison de l'adaptation des règles de compilation (CMakeLists.txt) pour l'écosystème Zephyr, une tâche qui n'avait pas été isolée dans la planification initiale. Néanmoins, ce surcroît de travail devrait être naturellement équilibré par la phase d'inventaire des fonctions, dont l'envergure s'avère plus restreinte qu'escompté.

Bibliographie

- [1] « Wi-SUN : Les réseaux sans fil intelligents et omniprésents expliqués ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://fr.digi.com/solutions/by-technology/wi-sun>
- [2] « Wi-SUN FAN Security Concepts and Design Considerations | Security Concepts and Design Considerations | Wi-SUN | v1.10.4 | Silicon Labs ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.silabs.com/wisun/1.10.4/wisun-security-concepts-design-considerations/>
- [3] « The Linux Foundation Announces Project to Build Real-Time Operating System for Internet of Things Devices | Zephyr Project ». Consulté le: 27 mars 2026. [En ligne]. Disponible sur: <https://web.archive.org/web/20160310073146/https://www.zephyrproject.org/news/linux-foundation-announces-project-build-real-time-operating-system-internet-things-devices>
- [4] « Mesh tutorial - API references and tutorials | Mbed OS 6 Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://os.mbed.com/docs/mbed-os/v6.16/apis/connectivity-tutorials.html>
- [5] « Important Update on Mbed | Mbed ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://os.mbed.com/blog/entry/Important-Update-on-Mbed/>
- [6] « CMSIS RTOS - Handbook | Mbed ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://os.mbed.com/handbook/CMSIS-RTOS>
- [7] « West (Zephyr's Meta-Tool) — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/develop/west/index.html>

- [8] « System Power Management — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/services/pm/system.html>
- [9] « Device Power Management — Zephyr Project Documentation ». Consulté le: 1 avril 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/services/pm/device.html>
- [10] « Important update on Mbed - End of Life - Mbed OS ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://forums.mbed.com/t/important-update-on-mbed-end-of-life/23644>
- [11] « Modules (External projects) — Zephyr Project Documentation ». Consulté le: 29 mars 2026. [En ligne]. Disponible sur: <https://docs.zephyrproject.org/latest/develop/modules.html>

Figures

Fig. 1 Planification Initiale	17
Fig. 2 Transaction driver -> Nanostack	25
Fig. 3 État de la planification au 02.04.2026	34

FIGURES

Tables

Tableau 1	Glossaire des termes techniques	7
Tableau 2	Comparaison des méthodes d'intégration logicielle sous Zephyr	28

Liste des extraits de code

Liste 1	Arborescence type du module Zephyr pour la Nanostack	29
Listing 2	Déclaration du module dans le west.yml de l'application	29
Listing 3	Fichier kconfig à la racine du module	30